UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"
Campus de São José dos Campos
Instituto de Ciência e Tecnologia

Linear Classifiers

Prof. Dr. Rogério Galante Negri

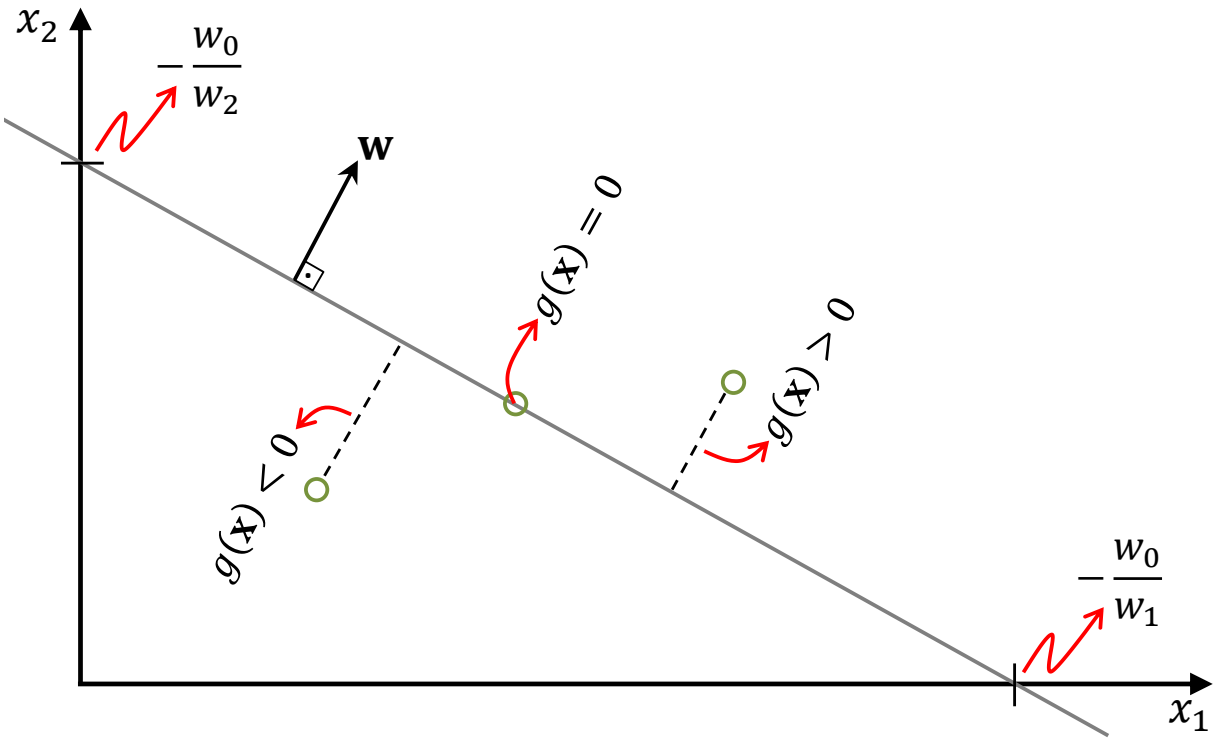
Agenda

Linear Classifiers and Linearly Separable Problems

- A *linear* classifier separates the classes using a linear decision surface
- Linear decision surface — the geometric locus that nullifies a linear discriminant function:

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0, \tag{1}$$

where $\mathbf{x}, \mathbf{w} \in \mathcal{X} \subseteq \mathbb{R}^n$ and $w_0 \in \mathbb{R}$



Linear Classifiers and Linear Problems

- It is important to consider the value of the discriminant function
- If $g(\mathbf{x}) > 0$, then $\mathbf{x}$ is located away from the surface $g(\mathbf{x}) = 0$ in the direction of $\mathbf{w}$
- Similarly, $g(\mathbf{x}) < 0$ indicates that the vector orthogonal to the decision surface, with endpoint at $\mathbf{x}$, has the opposite direction to $\mathbf{w}$
- The sign of $g(\mathbf{x})$ defines the decision rule of a linear classifier

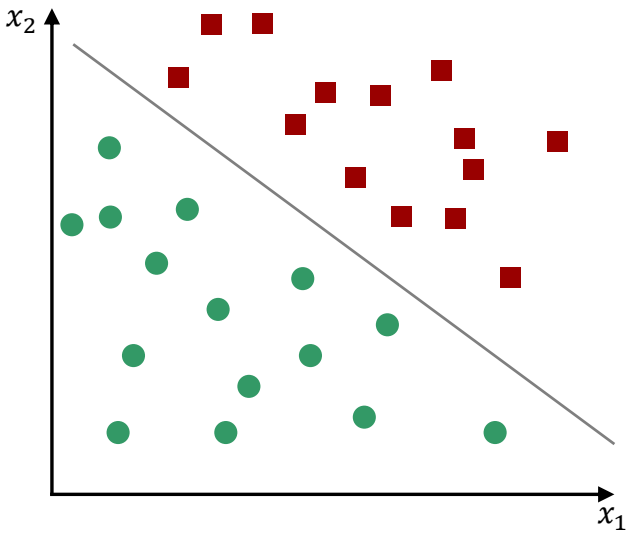
Decision rule for a linear classifier

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 \begin{cases} > 0 \Rightarrow \mathbf{x} \in \omega_1 \\ < 0 \Rightarrow \mathbf{x} \in \omega_2 \end{cases} \tag{2}$$

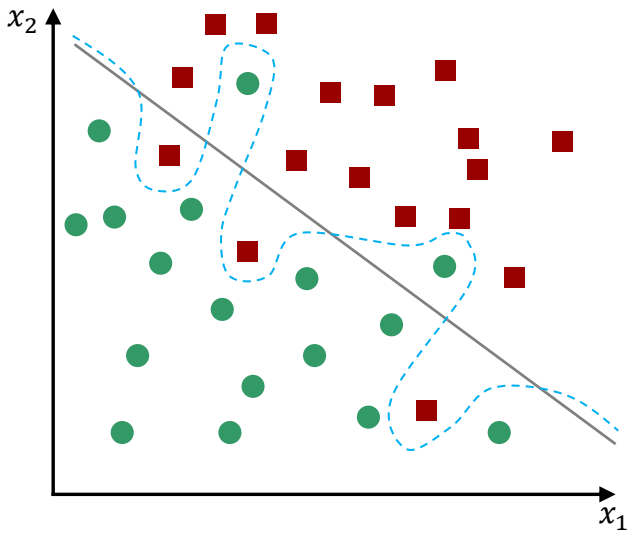
It is worth noting that a linear classifier can only separate two classes. Its use in problems involving more than two classes requires specific strategies...

Linear or Not!

- It is important to consider the value of the discriminant function
- If $g(\mathbf{x}) > 0$, then $\mathbf{x}$ is located away from the surface $g(\mathbf{x}) = 0$ in the direction of $\mathbf{w}$
- Similarly, $g(\mathbf{x}) < 0$ indicates that the vector orthogonal to the decision surface, with endpoint at $\mathbf{x}$, has the opposite direction to $\mathbf{w}$
- The sign of $g(\mathbf{x})$ defines the decision rule of a linear classifier



Linearly separable



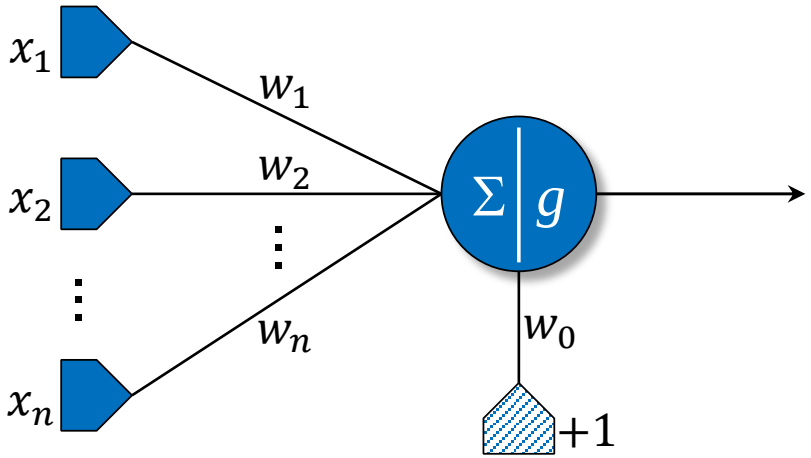
Non-linearly separable

For linearly separable data, there exists a linear surface capable of partitioning the feature space into two distinct class subsets.
This cannot be guaranteed for non-linearly separable data.

Agenda

Perceptron

- Proposed in 1958 by Frank Rosenblatt — a milestone in the development of Artificial Intelligence (tragic history)
- Inspired by the neuron model of Warren McCulloch and Walter Pitts
- It consists of five basic elements:
  - A layer of n receptors — receives the synaptic inputs $x_1, \ldots, x_n$
  - A set of synaptic connections — weights the inputs with weights $w_1, \ldots, w_n$
  - A constant input equal to $+1$, weighted by w_0 (i.e., bias)
  - An aggregator of the input signals that induces a synaptic field and forwards it to...
  - An activation function g, which generates a response



Perceptron (cont. 1)

- Given the set of observations $\mathcal{D} = \{(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \ldots, m\}$, where $\mathbf{x}_i$ is associated with ω_1 or ω_2 when y_i is $+1$ or -1, respectively
- Additionally, the classification problem occurs over an extended feature space, i.e.: $\mathbf{x} = [1, x_1, \ldots, x_n] \in \mathcal{X} \subseteq \mathbb{R}^{n+1}$
- Thus, the discriminant functions are reduced to $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ with $\mathbf{w} = [w_0, w_1, \ldots, w_n]$

The Perceptron searches for $\mathbf{w}$ such that $g(\mathbf{x}_i) > 0$ if $y_i = +1$, and $g(\mathbf{x}_i) < 0$ if $y_i = -1$. A solution to this problem can be found by minimizing:

$$J(\mathbf{w}) = \sum_{i=1}^{m} \left(\delta(\mathbf{x}_i; \mathbf{w})\mathbf{w}^T\mathbf{x}_i\right); \quad \delta(\mathbf{x}_i; \mathbf{w}) = \begin{cases} -1, & \text{if } y_i = +1 \text{ and } \mathbf{w}^T\mathbf{x}_i < 0; \\ +1, & \text{if } y_i = -1 \text{ and } \mathbf{w}^T\mathbf{x}_i > 0; \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Perceptron (cont. 2)

- The minimization of $J(\mathbf{w})$ is achieved via Gradient Descent
- Iterative strategy – successive updates to $\mathbf{w}$
- Let $\mathbf{w}^{(k)}$ be the k-th estimate of $\mathbf{w}$, we have:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta_k \left. \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}^{(m)}}, \tag{4}$$

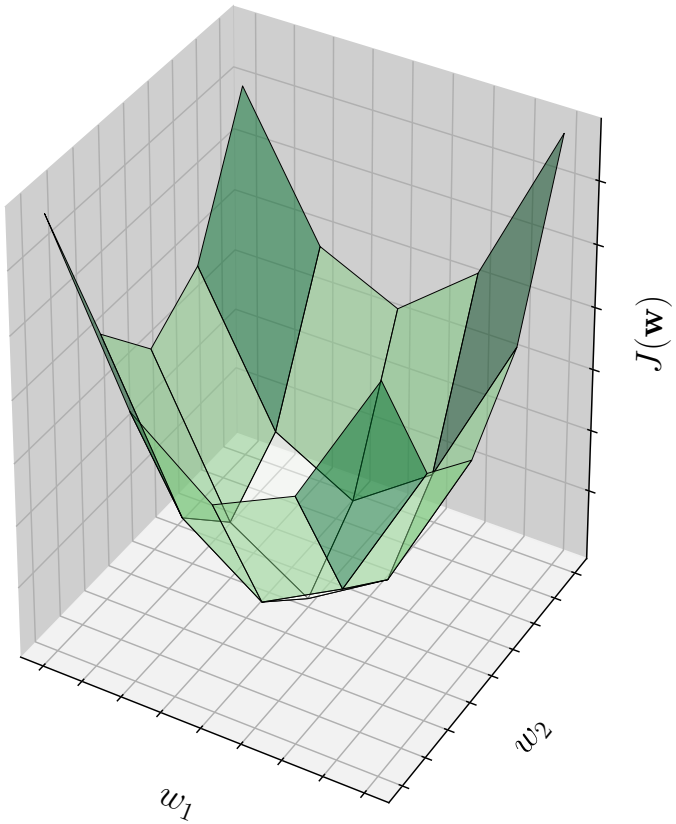
  where $\eta_k \in \mathbb{R}_+$ is a correction factor that controls the update of $\mathbf{w}^{(k)}$
- $\mathbf{w}^{(0)}$ is initialized randomly or with zeros
- Since $\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial \left(\sum_{i=1}^{m} \left(\delta(\mathbf{x}_i; \mathbf{w}) \mathbf{w}^T \mathbf{x}_i \right) \right)}{\partial \mathbf{w}} = \sum_{i=1}^{m} \delta(\mathbf{x}_i; \mathbf{w}) \mathbf{x}_i$

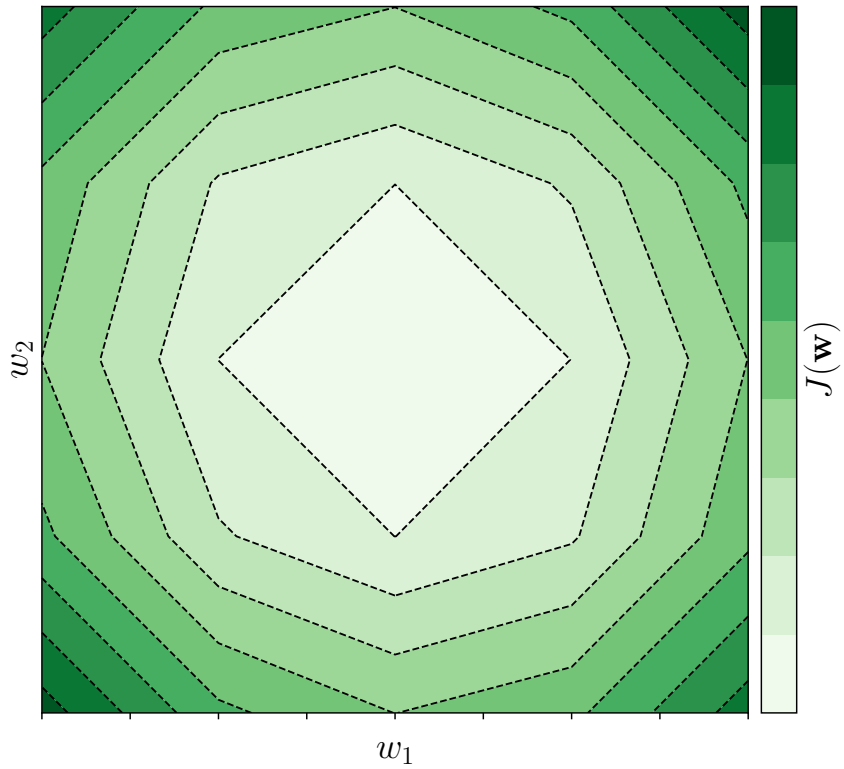
Perceptron Training

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta_k \sum_{i=1}^{m} \delta(\mathbf{x}_i; \mathbf{w}) \mathbf{x}_i, \tag{5}$$

Perceptron (cont. 3)

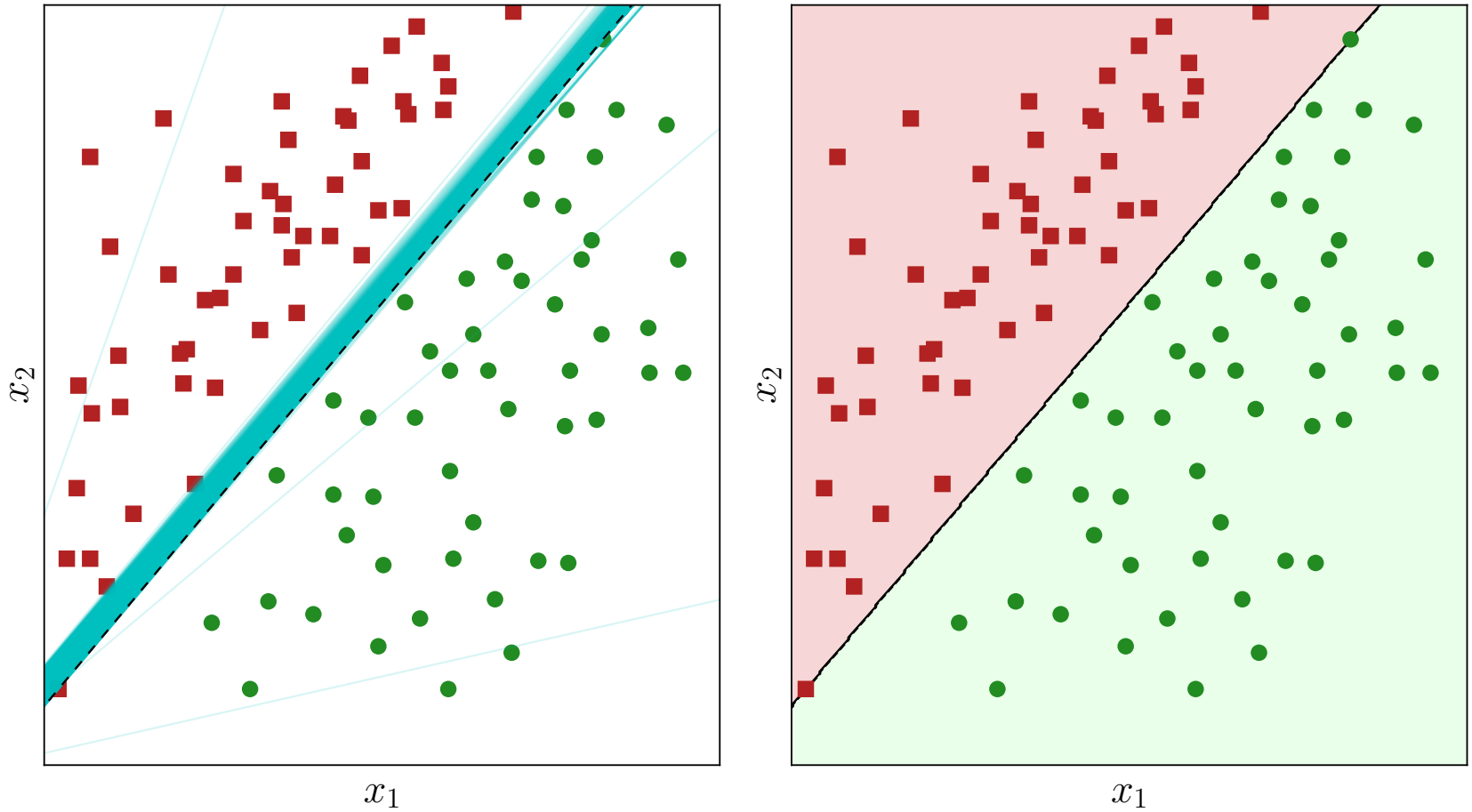
- As a consequence of the behavior of $\delta(\cdot;\cdot)$, the objective function that defines the Perceptron training becomes piecewise linear
- This behavior allows different synaptic weight configurations capable of minimizing the objective function





Perceptron (cont. 4)

Adjustment process of the decision surface by the Perceptron algorithm and final separation of the data.



The decision surfaces obtained throughout the iterations are indicated by green

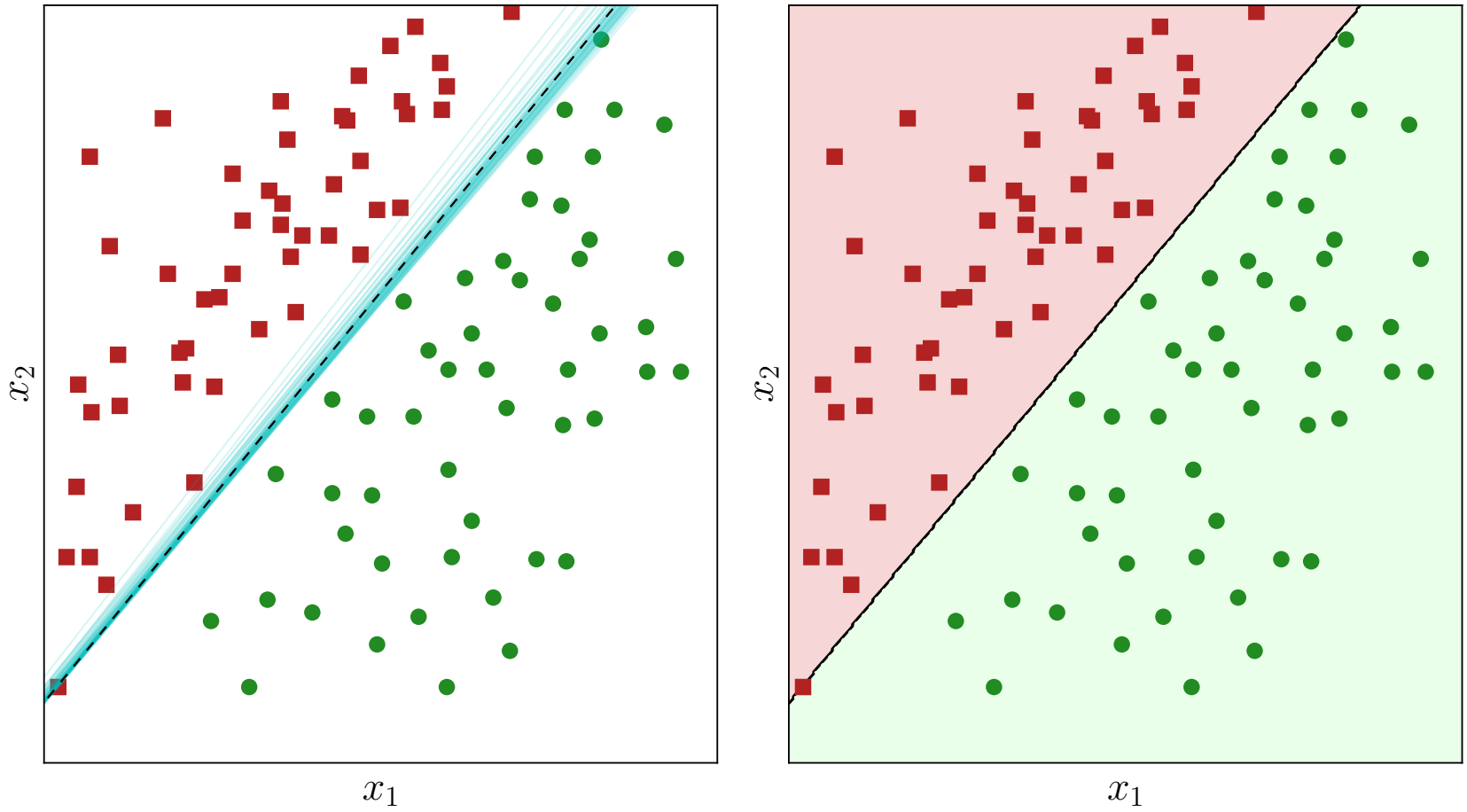
Sequential Perceptron

- The Perceptron searches for $\mathbf{w}$ iteratively
- The update of $\mathbf{w}$ is guided by $\sum_{i=1}^{m} \delta(\mathbf{x}_i; \mathbf{w})\mathbf{x}_i$ after applying the $\delta(\cdot;\cdot)$ function over all $\mathbf{x}_i$ in $\mathcal{D}$
- Variant of the original formulation — updates occur as each observation is presented, that is:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta_k \sum_{i=1}^{m} \delta(\mathbf{x}_i; \mathbf{w}^{(k)}). \tag{6}$$

- There is an update to $\mathbf{w}^{(k+1)}$ only when $\delta(\mathbf{x}_i; \mathbf{w}^{(k)})$, under the current parameterization $\mathbf{w}^{(k)}$, detects a classification error
- Despite the finite nature of $\mathcal{D}$, the observations $\mathbf{x}_i$ are presented successively through cycles

Sequential Perceptron (cont. 1)

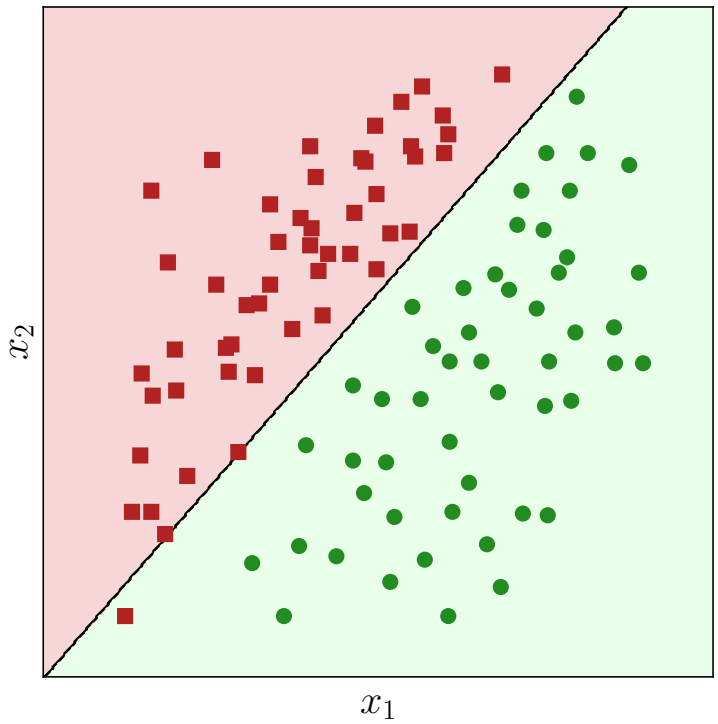


Decision surface obtained using the Perceptron algorithm, but updated sequentially.

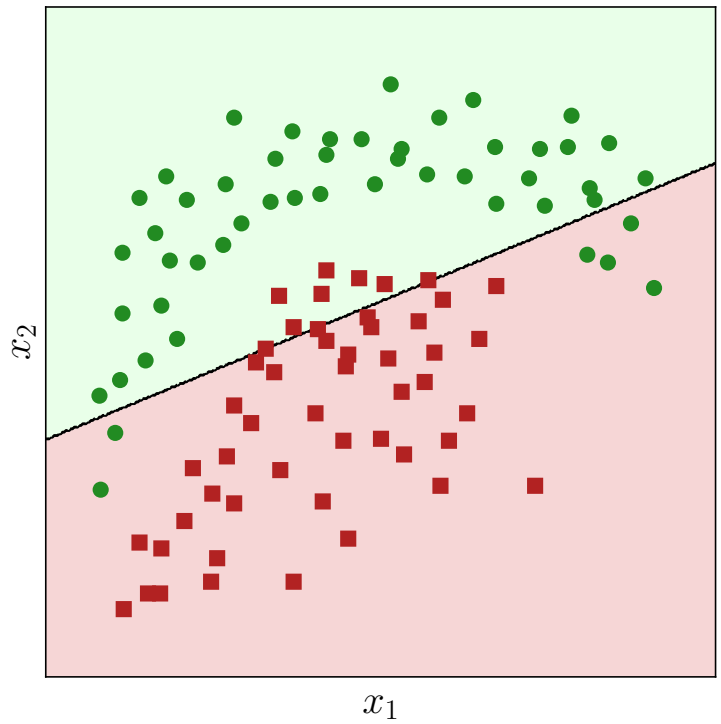
Pocket Perceptron for Non-Linearly Separable Data

- Initialize the counters $q, q^*, k \leftarrow 0$
- Initialize $\mathbf{w}^{(0)}$ and $\mathbf{w}^*$
- While not converged (zero error or maximum number of iterations), repeat:
- Apply $g(\mathbf{x}) = \mathbf{w}^{(k)^T}\mathbf{x}$ over each $\mathbf{x}_i$ in $\mathcal{D}$
- $q \leftarrow$ number of correct classifications using $\mathbf{w}^{(k)}$
- If $q > q^*$ then:
- $q^* \leftarrow q$
- $\mathbf{w}^* \leftarrow \mathbf{w}^{(k)}$
- $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta_k \sum_{i=1}^{m} \delta(\mathbf{x}_i; \mathbf{w}^{(k)})$
- $k \leftarrow k + 1$

Pocket Perceptron (cont.)



Linearly separable



Non-linearly separable

For linearly separable data, this strategy is almost reduced to the original version of the algorithm. On the other hand, in the case of non-linearly separable data, the algorithm stops after reaching the maximum number of iterations, and the solution is given by the **w** configuration that provides the highest accuracy among all tests performed.

Agenda

Sum of Squared Errors – SSE

- Starting from $\mathcal{D} = \{(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \ldots, m\}$
- Alternative linear classifier — modeled by the sum of squared errors
- Learning guided by $J(\mathbf{w})$, with $\mathbf{w} = (w_0, w_1, \ldots, w_n)$

$$J(\mathbf{w}) = \sum_{i=1}^{m} \left(y_i - \mathbf{w}^T \mathbf{x}\right)^2 \tag{7}$$

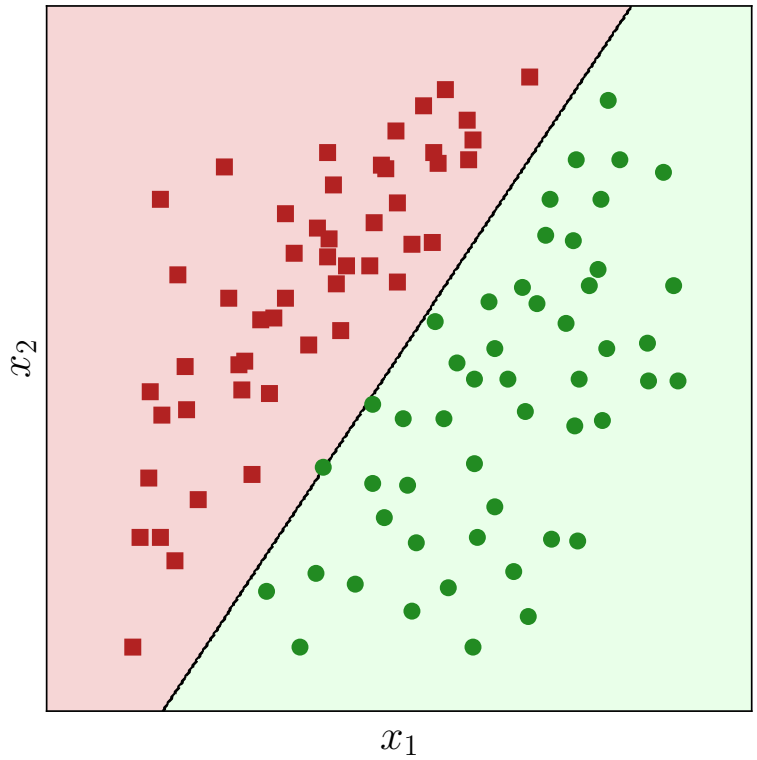
Minimizing $J(\mathbf{w})$ leads to $A = \sum_{i=1}^{m} \mathbf{x}_i \mathbf{x}_i^T$ and $B = \sum_{i=1}^{m} y_i \mathbf{x}_i^T$

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = 0 \Leftrightarrow \frac{\partial \sum_{i=1}^{m} \left(y_i - \mathbf{w}^T \mathbf{x}_i\right)^2}{\partial \mathbf{w}} = 0 \Leftrightarrow -2 \sum_{i=1}^{m} y_i \mathbf{x}_i^T + 2 \sum_{i=1}^{m} \mathbf{x}_i \mathbf{x}_i^T \mathbf{w} = 0 \Leftrightarrow$$

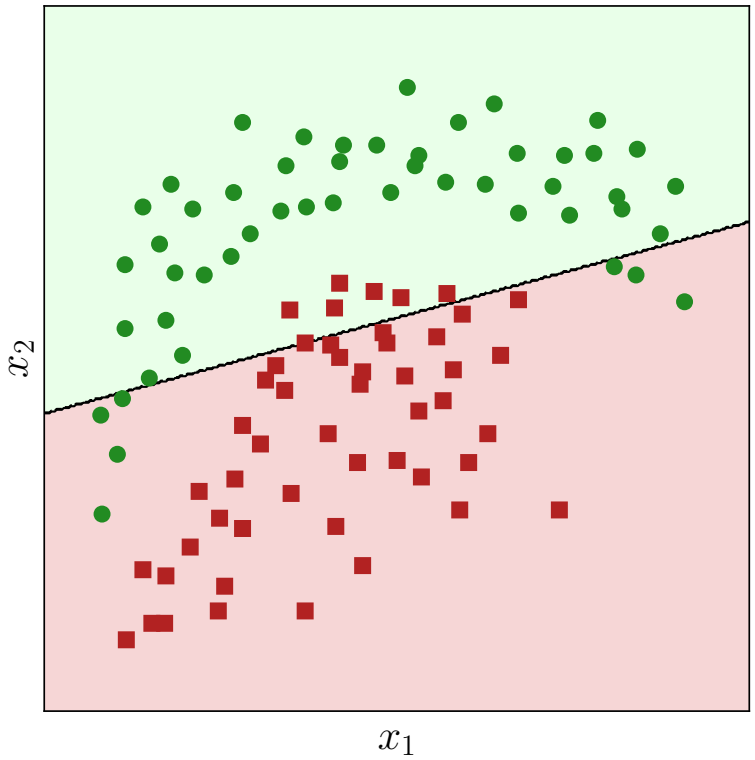
$$\Leftrightarrow \mathbf{w} \sum_{i=1}^{m} \mathbf{x}_i \mathbf{x}_i^T = \sum_{i=1}^{m} y_i \mathbf{x}_i^T \tag{8}$$

Finally, we obtain $\mathbf{w} = A^{-1} B$

Sum of Squared Errors (cont. 1)



Linearly separable



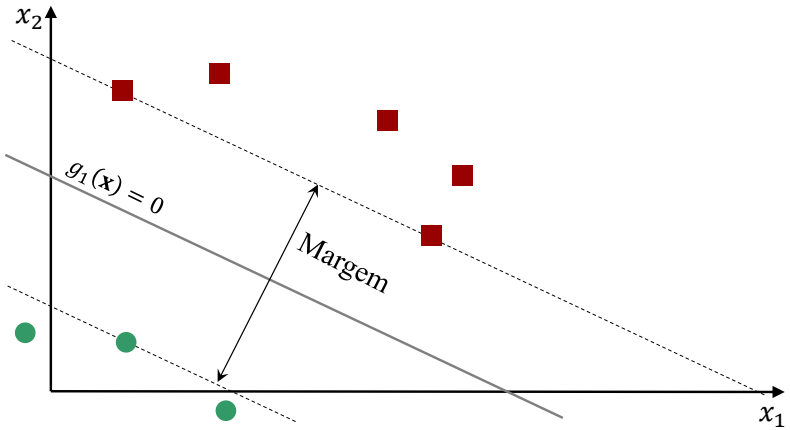
Non-linearly separable

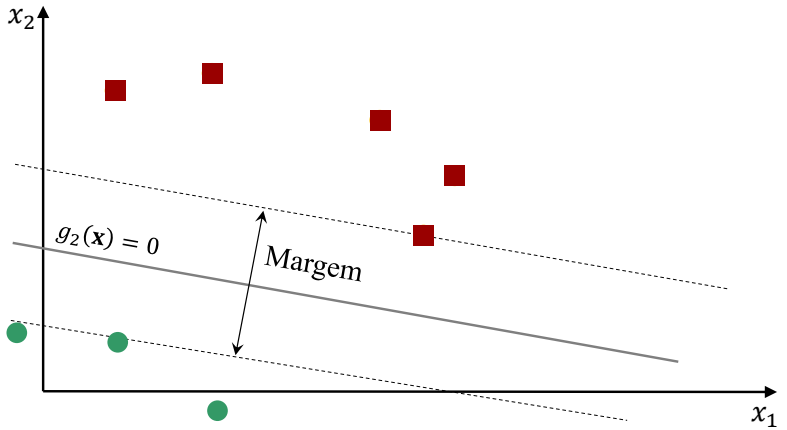
Using this technique yields results similar to those provided by the Perceptron algorithm associated with the relaxation strategy.

Agenda

Support Vector Machine

- Support Vector Machine (SVM) has become prominent as a pattern recognition method with broad application in various research fields
- Strong theoretical foundation, independence from statistical models, simple architecture, moderate computational complexity, and excellent generalization capability
- It seeks a linear discriminant function g that classifies $\mathcal{D} = \{(\mathbf{x}_i, y_i) : i = 1, \ldots, m\}$ as belonging to ω_1 when $y_i = +1$, and ω_2 when $y_i = -1$
- Discriminant function that provides the maximum separation margin between the training examples





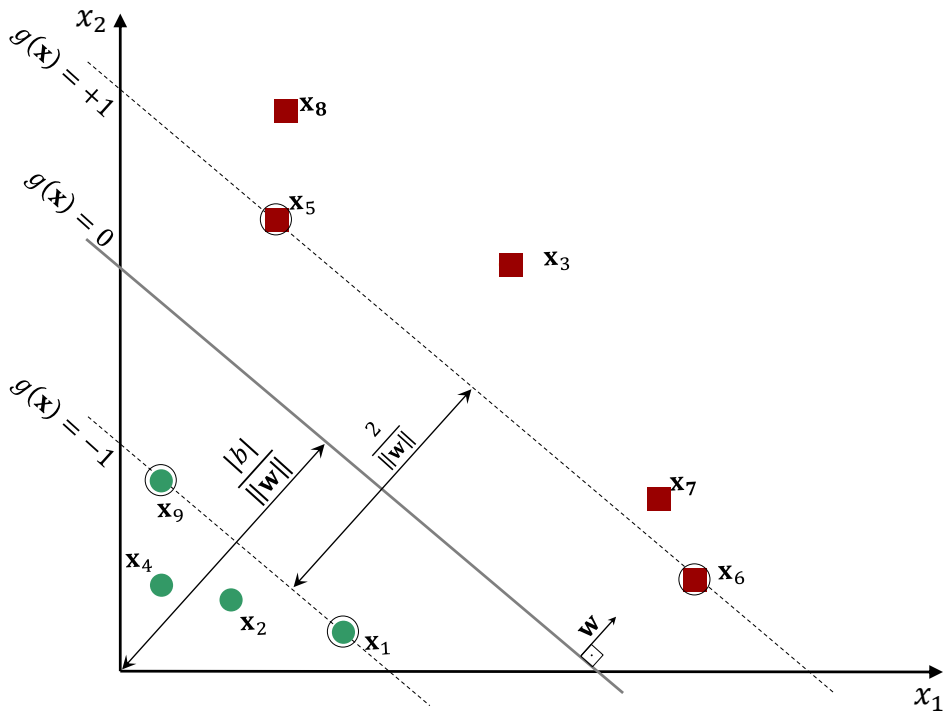
SVM — Linearly Separable Case

- The distance between any pattern (point) $\mathbf{x}_i$ and the separating hyperplane (line) $g(\mathbf{x}) = 0$ is given by $\frac{|g(\mathbf{x}_i)|}{\|\mathbf{w}\|}$
- Rescaling $\mathbf{w}$ such that two patterns $\mathbf{x}_u \in \omega_1$ and $\mathbf{x}_v \in \omega_2$, both in $\mathcal{D}$ and closest to $g(\mathbf{x}) = 0$, are at unit distance from the hyperplane, leads to the following:
  1. The margin width is $\frac{1}{\|\mathbf{w}\|} + \frac{1}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}$
  2. Given $\mathbf{x}_u \in \omega_1$, then $g(\mathbf{x}_u) = \mathbf{w}^T\mathbf{x}_u + b \geq +1$
  3. Given $\mathbf{x}_v \in \omega_2$, then $g(\mathbf{x}_v) = \mathbf{w}^T\mathbf{x}_v + b \leq -1$

Based on these conditions, the following optimization problem is modeled. Its solution leads to the parameters $\mathbf{w}$ and b that define the maximum-margin hyperplane:

$$\begin{aligned} &\min_{\mathbf{w},b} \; J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^T\mathbf{w} \\ &\text{subject to: } \; y_i\left(\mathbf{w}^T\mathbf{x}_i + b\right) \geq 1, \;\; i = 1, \ldots, m \end{aligned} \tag{9}$$

SVM — Linearly Separable Case (cont. 1)



Geometric relationships that the maximum-margin hyperplane must satisfy — the patterns located on the boundaries of the margin (i.e., $g(\mathbf{x}) = \pm 1$) determine the optimal hyperplane — These patterns are called support vectors.

SVM — Linearly Separable Case (cont. 2)

- It is convenient to solve the optimization problem using the method of Lagrange multipliers, reformulated as:

$$\begin{aligned} &\max_{\lambda} L_D(\lambda) = \sum_{i=1}^{m} \lambda_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ &\text{subject to: } \begin{cases} \lambda_i \geq 0, \ i = 1, \ldots, m \\ \sum_{i=1}^{m} \lambda_i y_i = 0 \end{cases} \end{aligned} \tag{10}$$

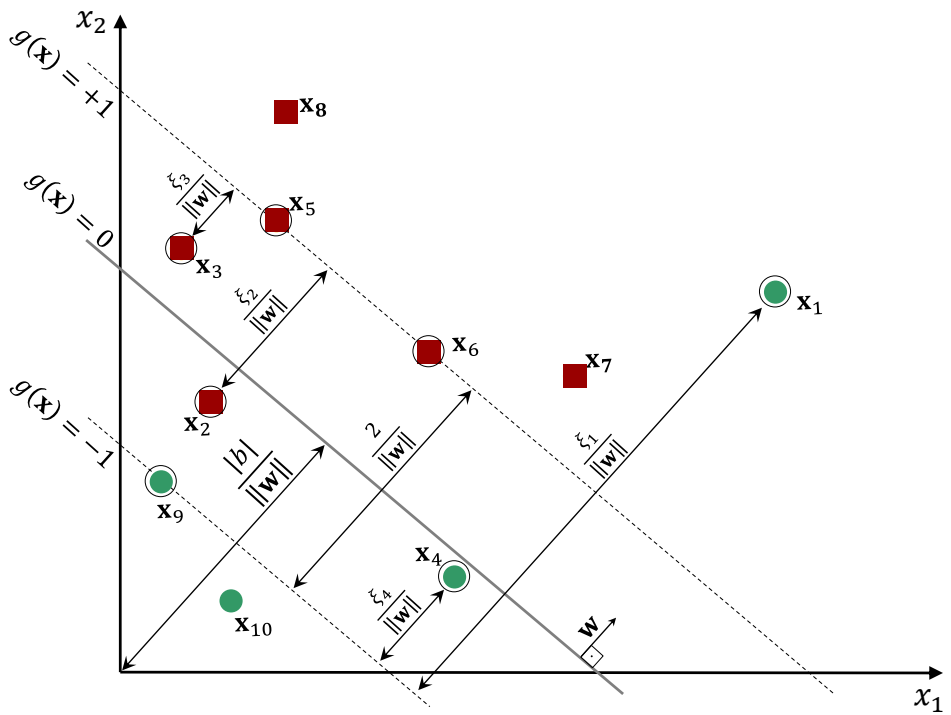
- The Lagrange multipliers obtained from the solution above allow the calculation of the parameters:
  - $\mathbf{w} = \sum_{i=1}^{m} \lambda_i y_i \mathbf{x}_i$
  - $b = \mathbf{w}^T \mathbf{x}_i - 1$, where $\mathbf{x}_i$ is a support vector with $y_i = 1$
- Therefore, we have:

$$g(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b \begin{cases} \geq 0 \Rightarrow x_i \in \omega_1 \\ < 0 \Rightarrow x_i \in \omega_2 \end{cases} \tag{11}$$

SVM — Non-Linearly Separable Case

It is natural that some problems involve patterns that are not linearly separable — to address this, slack variables $\xi \geq 0$ are introduced to ensure the following conditions are always true:

1. Given $\mathbf{x}_i \in \omega_1$, then $g(\mathbf{x}_i) = \mathbf{w}^T\mathbf{x}_i + b \geq +1 - \xi_i$
2. Given $\mathbf{x}_i \in \omega_2$, then $g(\mathbf{x}_i) = \mathbf{w}^T\mathbf{x}_i + b \leq -1 + \xi_i$



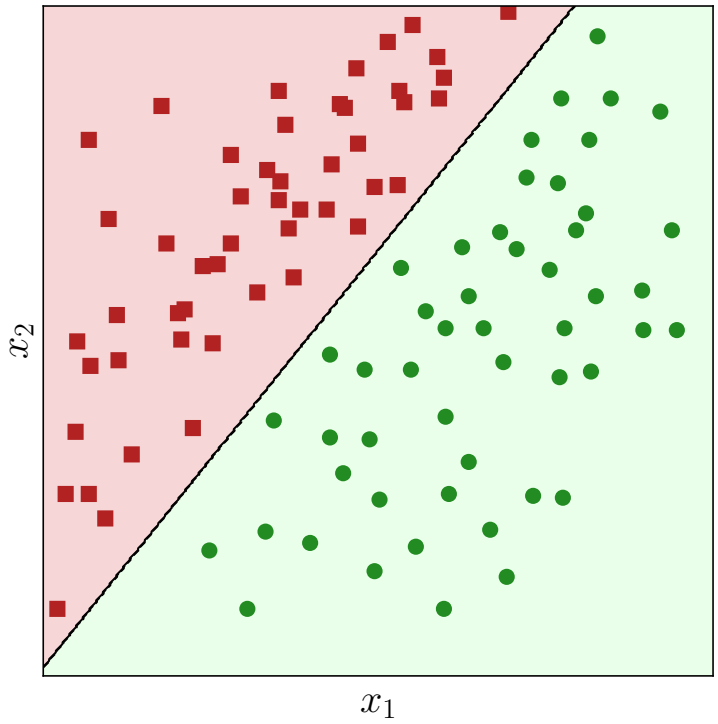
SVM – Non-Linearly Separable Case (cont.1)

- After proper reformulation, the dual form of the Lagrangian function for the non-separable case becomes identical to that of the linearly separable case
- The only difference lies in the constraints, as follows:

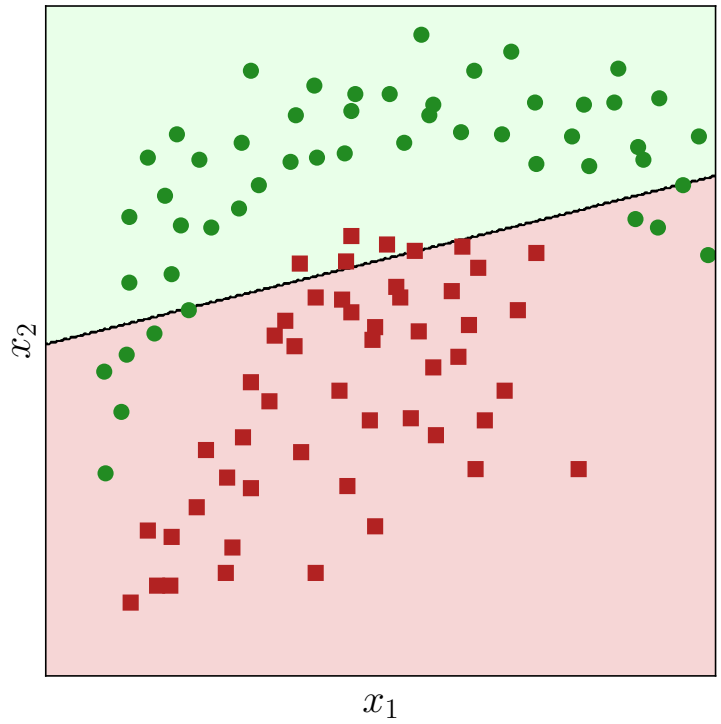
$$\begin{aligned} &\max_{\lambda} L_D(\lambda) = \sum_{i=1}^{m} \lambda_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ &\text{subject to: } \begin{cases} 0 \leq \lambda_i \leq C, \quad i = 1, \ldots, m \\ \sum_{i=1}^{m} \lambda_i y_i = 0 \end{cases} \end{aligned} \tag{12}$$

- The parameters $\mathbf{w}$ and b are obtained analogously, as well as the classification process via $g(\mathbf{x})$

SVM



Linearly separable



Non-linearly separable

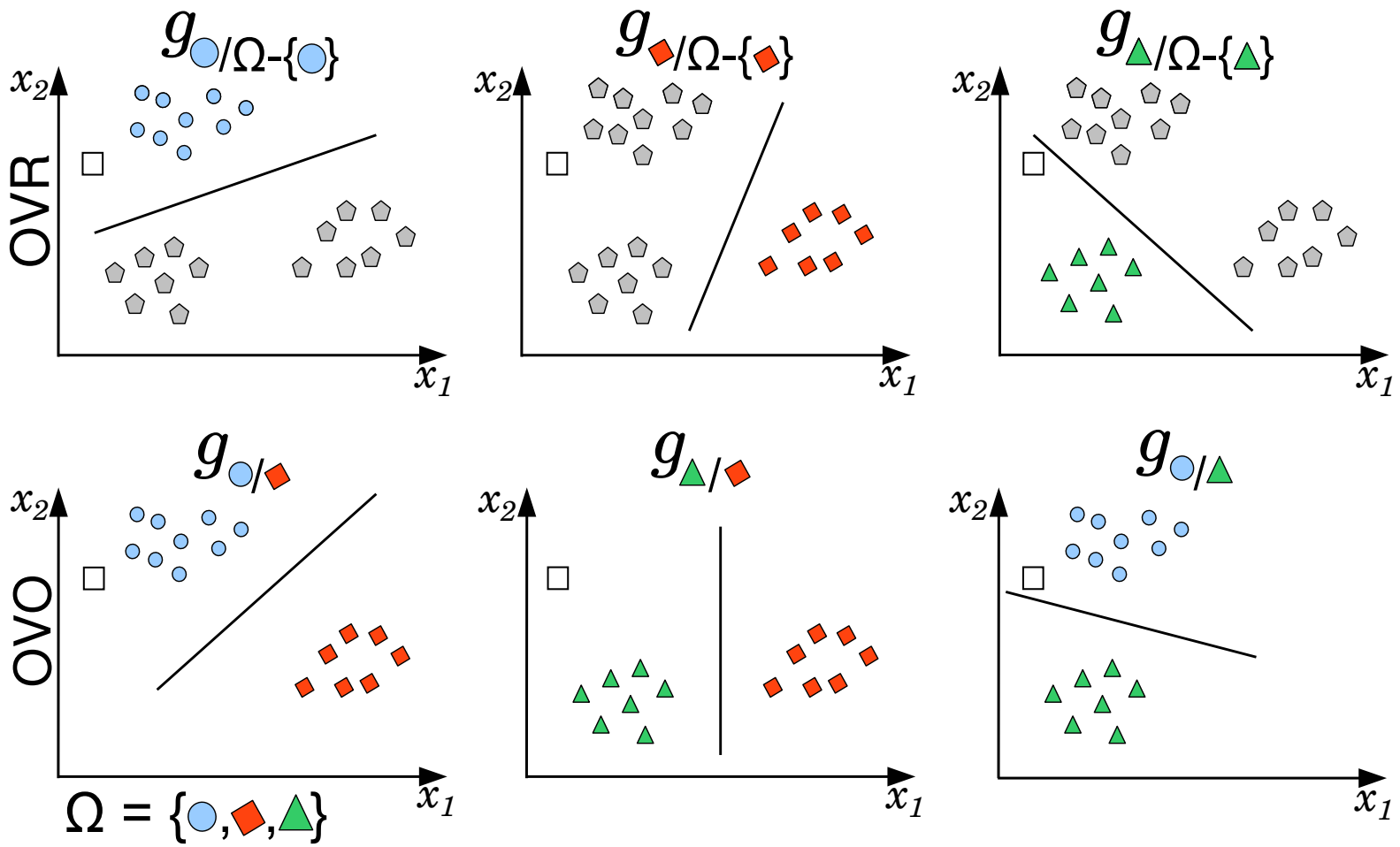
In the linearly separable case, we observe that the separating surface is further from the training examples near the decision boundary compared to the methods discussed earlier.
Its application to non-linearly separable data produces decision surfaces similar to those obtained by the previously discussed methods.

Agenda

Multiclass Strategies

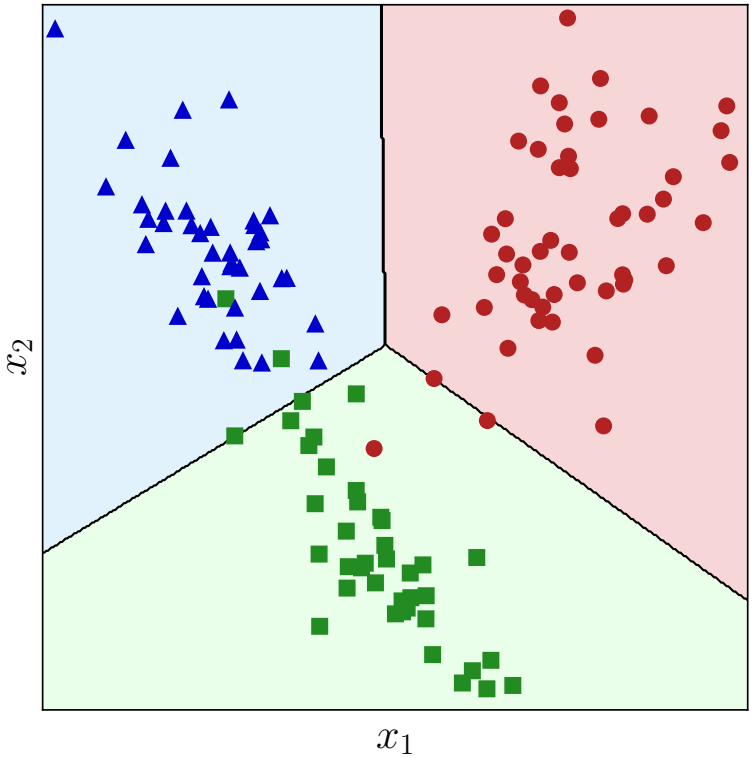
- Previous formalisms (Perceptron, SSE, SVM) — binary classification
- Practical problems often involve more than two classes
- To overcome this limitation, multiclass strategies are employed:
  - Decomposition into binary subproblems (preferred)
  - Reformulation of the method
- Common decomposition strategies — One-Versus-Rest (OVR) and One-Versus-One (OVO)
- OVR — for a problem with c classes, define c binary classifiers. Each classifier separates one specific class from the rest.
- OVO — for a problem with c classes, define $c(c-1)/2$ binary classifiers — each separates a pair of classes, and the final decision is made by majority voting.

Multiclass Strategies (cont.1)

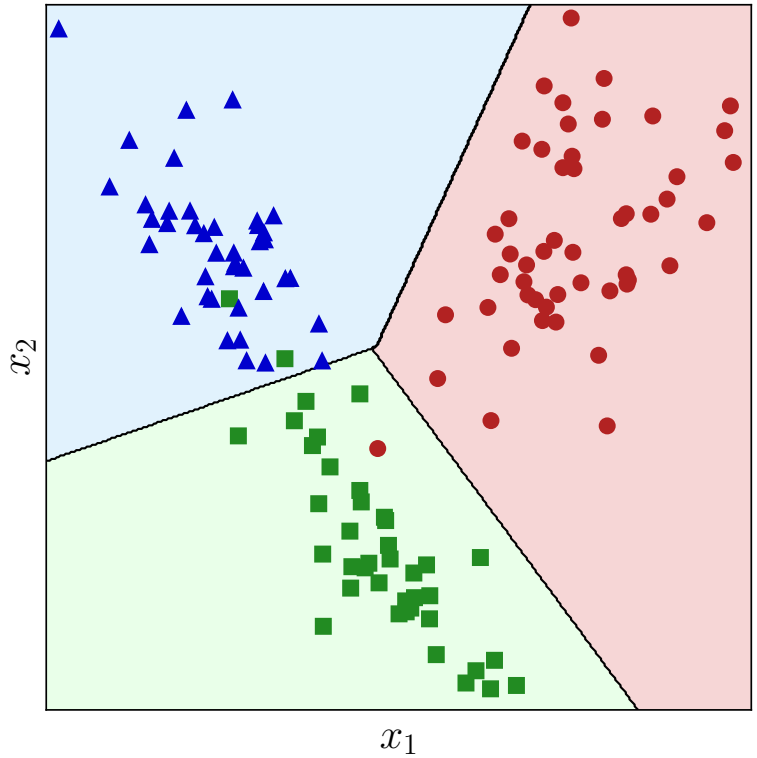


Example of applying OVR and OVO multiclass strategies to a three-class separation problem.

Multiclass Strategies (cont.2)



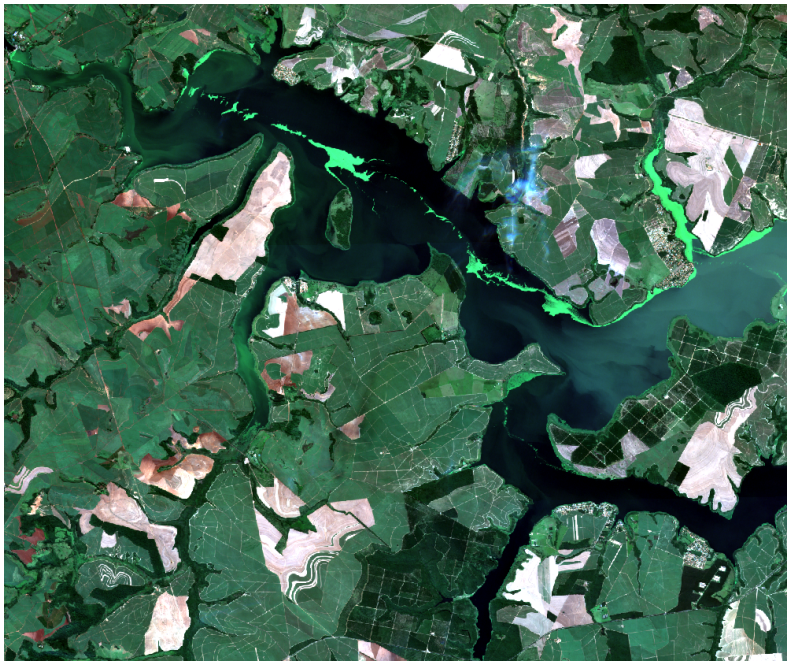
SSE



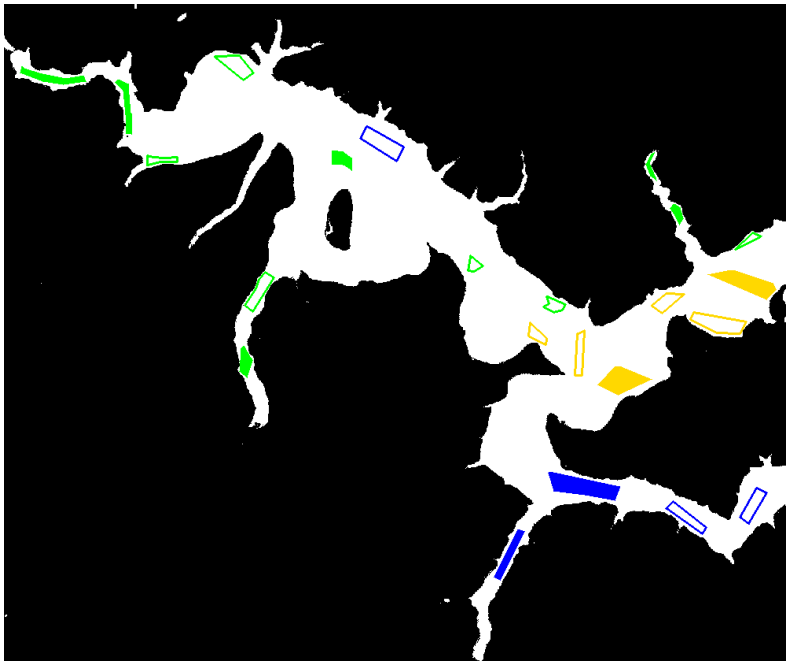
SVM ($C = 100$)

Agenda

Computational Experiment: Linear Separators for Binary and Multiclass Classification



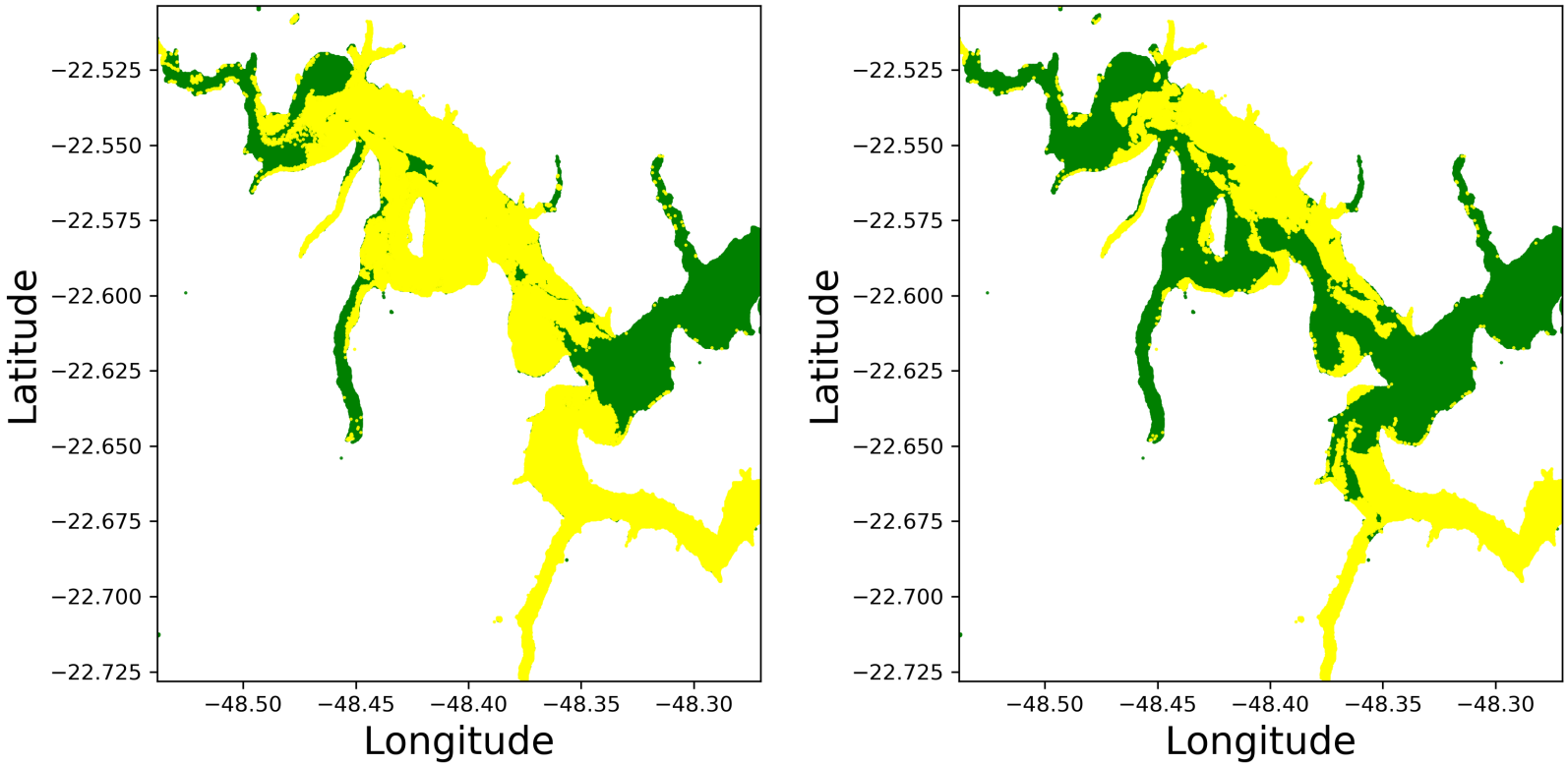
(a) Image used in the experiment



(b) Target object and training samples

Figura: Data used in the experiment. Only the feature vectors of pixels within the region of interest (white area — left) are classified. Three classes are identified: severe algae occurrence (green); moderate occurrence (yellow); and absence/low occurrence (blue).

Computational Experiment: Linear Separators for Binary Classification

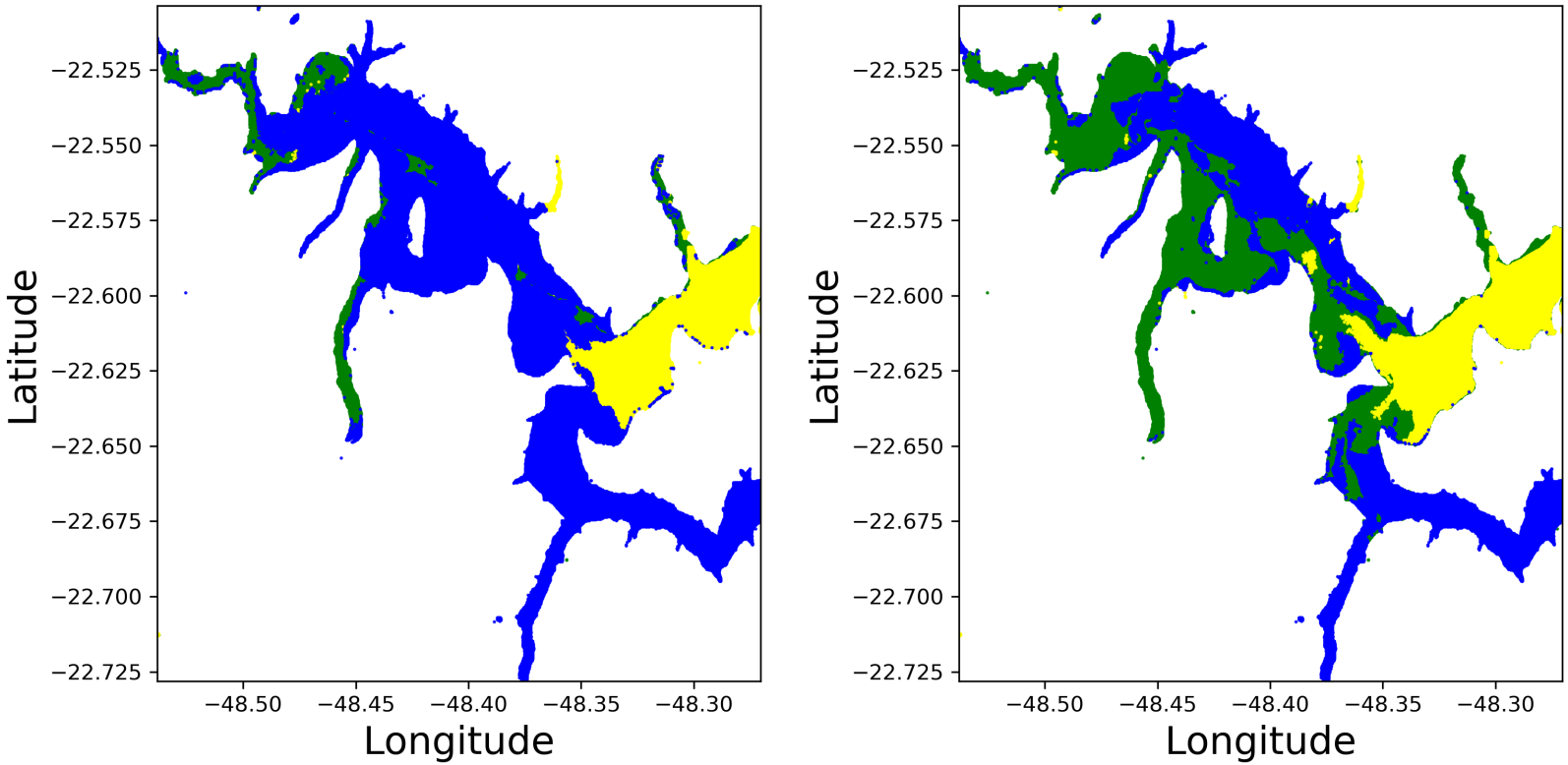


(a) SSE — Accuracy: 97.7%

(b) SVM ($C = 100$) — Accuracy: 99.8%

Figura: Results for the binary classification problem involving severe (green) and moderate+absent (yellow) algae occurrence.

Computational Experiment: Linear Separators for Multiclass Classification



(a) SSE — Accuracy: 95.7%

(b) SVM — Accuracy: 99.6%

Figura: Results for the multiclass classification problem involving: severe algae occurrence (green), moderate (blue), and absent (blue).

Lineares Classifiers

rogerio.negri@unesp.br